PE3: tareas para delegar

Oct 26, 2026 · @Maisa

Contexto

Tres tareas independientes del modelo para la PE3, que se entrega el jueves 5/11. Cada una se le entrega a Gonzalo en la fecha que figura en su sección, y Gonzalo la integra al repo, así llegamos a usarla en el análisis y en el dashboard. Nadie sube nada a `main` directamente.

Qué pide la PE3. Pasar de la exploración a un análisis validado: evaluar las hipótesis con herramientas estadísticas, aplicar técnicas avanzadas con su validación, traducir los resultados en una regla de decisión y armar un dashboard interactivo para el analista del fondo. El repositorio tiene que quedar más depurado y reproducible. La consigna completa y la devolución de la PE2 están en el mail del profe.

Qué hacemos Gonzalo y Claude en paralelo. El segundo relevamiento de MercadoLibre, el leakage indirecto, la estabilidad de las subzonas, la comparación de modelos con intervalos de predicción, los tests de H1, H2 y H3, el embudo de universos del ranking, la regla de decisión y el dashboard en HTML. Ninguna de las tres tareas de abajo depende de eso, y nada de eso depende de ellas hasta la integración.

Material de referencia, todo en el repo:

- `README.md`: estado de la PE2, sobre todo la sección 7 (fuentes) y la 11 (pendientes).
- `docs/entregas_grupo/fuentes_externas.md`: formato de las fichas de fuentes.
- `docs/entregas_grupo/preguntas_y_alcance.md`: qué no mide el análisis y el borrador de la regla de decisión.
- `docs/pipeline.md` y `config/config.yaml`: cómo corre el pipeline.

Tarea 1: tests formales y permisos de obra

Dos partes: pasar la verificación del pipeline a pytest con integración continua, y sumar los permisos de obra de BA Data como señal de dinamismo de la zona. El profe pidió "completar los tests formales" y los permisos estaban como pendiente en la sección 11 del README.

Fechas (entrega a Gonzalo): permisos el viernes 30/10 a la noche; tests el domingo 1/11 a la noche.

Parte A: tests con pytest

Hoy existe `tests/verificar_offline.py`, un script que compara por hash las salidas de las etapas 1 a 4 contra los datasets versionados. Funciona, pero no es una suite de tests formal.

1. Pasar cada control del script a una función `test_...` de pytest en `tests/test_pipeline.py`, sin perder lo que verifica (hashes, parseo de fixtures, replay de USIG).
2. Sumar tests unitarios de funciones puras, con casos chicos armados a mano:
   - `src/limpieza.py`: `corregir_precio`, `parsear_superficie`, `parsear_antiguedad`, `parsear_expensas`, `marcas_del_titulo`, `outliers_iqr_por_grupo`.
   - `src/geocoding_propiedades.py`: `limpiar_direccion`.
   - `src/enriquecimiento.py`: `haversine_np` (una distancia conocida, por ejemplo Obelisco a Plaza de Mayo) y `parsear_estaciones`.
   - `src/eda.py`: `precio_relativo` y `error_vara_fuera_de_muestra`.
   - Los tests de la Parte B.
3. Un workflow de GitHub Actions (`.github/workflows/tests.yml`) que instale `requirements.txt` y corra `pytest` en cada push y pull request. Sumar `pytest` a `requirements.txt` con versión fijada.
4. Una sección corta en `docs/pipeline.md`: cómo correr los tests y qué cubre cada archivo.

No testear `src/modelo.py`: lo estamos cambiando nosotros. Lo sumamos al final.

Terminado cuando: `pytest` pasa en limpio desde la raíz del repo y cada test tiene un nombre que dice qué verifica.

Parte B: permisos de obra

Un aviso en una zona con muchas obras nuevas está en un entorno que se está valorizando, y eso puede explicar parte de su precio o de su liquidez.

1. Buscar en BA Data el dataset de permisos de obra (registrados o en ejecución) con coordenadas. Elegir un período razonable (por ejemplo, los últimos 3 a 5 años) y justificarlo.
2. Guardar el snapshot en `data/external/` y escribir un módulo nuevo `src/permisos_obra.py` que, para cada aviso con coordenadas, calcule:
   - `permisos_obra_500m`: cantidad de permisos a menos de 500 m.
   - `m2_obra_500m`: superficie total autorizada a menos de 500 m, si el dataset la trae.

3. **No modificar `src/enriquecimiento.py` ni los datasets existentes.** La salida va a un archivo aparte, `data/processed/permisos_obra.tsv`, con la columna `link` como clave y las columnas nuevas. Nosotros lo unimos al dataset de trabajo. Así no se rompen los hashes de los tests ni se mueven nuestros números a mitad de camino.
4. Agregar la ficha de la fuente en `docs/entregas_grupo/fuentes_externas.md`, con el mismo formato que las demás, y una tabla corta con la distribución de `permisos_obra_500m` por barrio.

Terminado cuando: `python3 src/permisos_obra.py` genera el TSV desde el snapshot sin red, tiene una fila por aviso del dataset enriquecido y tiene sus tests en la Parte A.

Tarea 2: fuentes externas para H3

Sumar tren, Metrobus, espacios verdes, barrios oficiales y barrios populares, para volver a evaluar H3 con una accesibilidad que cubra el sur y el oeste. Es la prioridad más fuerte que marcó el profe: en la PE2 solo teníamos subte, que casi no llega a esos barrios.

Fecha (entrega a Gonzalo): el viernes 30/10 a la noche. El sábado corremos H3 con estos datos.

Qué hay que calcular

Para cada aviso con coordenadas (`geo_status == "OK"` en `data/processed/propiedades_enriquecidas.tsv`):

Columna	Fuente (BA Data)	Cálculo
`dist_tren_m`	Estaciones de ferrocarril	Distancia en metros a la estación más cercana
`dist_metrobus_m`	Estaciones o paradas de Metrobus	Distancia a la parada más cercana
`dist_transporte_pesado_m`	Subte, tren y Metrobus juntos	Mínimo de las tres distancias (subte ya está en `dist_transporte_m`)
`dist_espacio_verde_m`	Espacios verdes (polígonos)	Distancia al borde del espacio verde más cercano; 0 si el aviso cae adentro
`area_verde_500m_ha`	Espacios verdes	Hectáreas de espacio verde a menos de 500 m. Una plaza chica no es lo mismo que un parque

Columna	Fuente (BA Data)	Cálculo
`barrio_oficial`	Polígonos de barrios	Barrio oficial donde cae el punto
`en_barrio_popular`	Barrios populares (polígonos)	1 si el punto cae adentro de un barrio popular, 0 si no

Para las distancias a puntos pueden reutilizar `haversine_np` de `src/enriquecimiento.py`. Para los polígonos conviene `geopandas` (sumarlo a `requirements.txt` con versión fijada). Ojo con las proyecciones: para medir distancias y áreas en metros hay que pasar a una proyección métrica, no medir en grados.

Cómo entregarlo

1. Bajar cada fuente a `data/external/` como snapshot, con la fecha de descarga en la ficha. El módulo tiene que correr sin red si el snapshot está.
2. Un módulo nuevo, `src/fuentes_externas.py`, que lea el dataset enriquecido y escriba `data/processed/accesibilidad.tsv`: columna `link` como clave, una fila por aviso del dataset enriquecido, y las columnas de la tabla.
3. **No modificar `src/enriquecimiento.py` ni los datasets existentes.** Nosotros unimos el archivo al dataset de trabajo.
4. Fichas de cada fuente en `docs/entregas_grupo/fuentes_externas.md`, con el mismo formato que las demás, (el estado de cada fuente en el README lo actualizamos nosotros al integrar).

Controles que tienen que reportar

Un archivo corto `docs/entregas_grupo/control_fuentes.md` con:

- Cantidad de estaciones o polígonos de cada fuente.
- Mediana de cada distancia por barrio, en una tabla. Mirar especialmente sur y oeste (Villa Lugano, Mataderos, Villa Soldati, Liniers): la distancia a transporte pesado tendría que bajar mucho respecto de solo subte.
- **Barrio de MercadoLibre contra barrio oficial.** Tabla de cruce: cuántos avisos caen en un barrio oficial distinto del de la búsqueda, y en qué pares (por ejemplo Barrio Norte contra Recoleta, que es esperable). Esto resuelve la observación del profe sobre los duplicados con discrepancia territorial.
- Cuántos avisos caen en barrios populares y en qué barrios (en la PE2 apareció Rodrigo Bueno en Puerto Madero).

Terminado cuando: `python3 src/fuentes_externas.py` genera `accesibilidad.tsv` desde los snapshots, el control está escrito y las fichas están completas. Si la Tarea 1 ya tiene pytest andando, sumar dos o tres tests de este módulo (un punto adentro de un polígono conocido, una distancia conocida).

Tarea 3: costos y liquidez para la regla de decisión

Juntar, con fuentes, los parámetros que convierten la brecha de precio en una brecha neta: cuánto cuesta comprar, cuánto cuesta refaccionar y qué tan rápido se vende en cada zona. El profe pidió "incorporar costos de refacción, liquidez, estado y riesgo legal" y "traducir las señales en una regla de decisión para el fondo". Nosotros armamos la regla; esta tarea le pone números defendibles.

Fecha (entrega a Gonzalo): el viernes 30/10 a la noche.

Qué hay que encontrar

Para cada parámetro: valor central, rango (mínimo y máximo razonables), fuente con link y fecha. Un rango con fuente sirve más que un número exacto sin fuente, porque vamos a hacer análisis de sensibilidad.

Bloque	Parámetro	Unidad
Costos de entrada	Honorarios de escribano	% del precio
	Impuesto de sellos en CABA (y cuándo hay exención; un fondo no compra vivienda única)	% del precio
	Comisión inmobiliaria del comprador	% del precio
	Otros gastos de escritura (certificados, inscripción)	% o USD
Refacción	Refacción liviana (pintura, terminaciones)	USD por m²
	Refacción media (baño y cocina)	USD por m²
	Refacción integral (instalaciones)	USD por m²
Negociación	Brecha entre precio publicado y precio de cierre (hoy usamos ~5%)	%
Liquidez	Tiempo promedio de venta en CABA, y por zona si existe	meses
	Escrituras de compraventa por mes en CABA (Colegio de Escribanos)	cantidad

Fuentes posibles: Colegio de Escribanos de CABA, AGIP (sellos), Colegio Inmobiliario de CABA, Índice de la Cámara Argentina de la Construcción o de revistas de obra para costos por m², informes de Reporte Inmobiliario o Zonaprop. Si una fuente está en pesos, convertir con el dólar MEP de la fecha y anotarlo.

Qué entregar

1. `config/costos.yaml`: un bloque por parámetro con `valor`, `minimo`, `maximo`, `unidad`, `fuente` y `fecha`. Así el análisis lee los números de un solo lugar.
2. `docs/entregas_grupo/costos_y_liquidez.md`: una página con la tabla de valores y, para cada bloque, dos o tres líneas sobre de dónde sale el número y qué tan confiable es.
3. **Mapeo de estado a refacción.** El dataset tiene marcas del título ("a refaccionar", "reciclado" y otras; ver `marcas_del_titulo` en `src/limpieza.py`). Proponer qué nivel de refacción corresponde a cada marca, y cuál asumir cuando el aviso no dice nada.
4. **Riesgo legal.** Una lista corta de palabras del título o la descripción que indiquen riesgo (sucesión, con inquilino, ocupado, sin escritura, subasta, crédito pendiente), con la razón de cada una. No hace falta programarlo: nosotros lo pasamos a código.

Terminado cuando: cada parámetro de la tabla tiene valor, rango, fuente y fecha, o una línea que explica por qué no se encontró. Si sobra tiempo, ayudar con los tests de la Tarea 1.

Reglas comunes

Cada uno trabaja en su copia del repo y le entrega todo a Gonzalo, que lo integra. Así no se pisan los cambios entre tareas ni con lo nuestro.

- **Qué entregar:** un `.zip` con los archivos nuevos (código, snapshots de datos, `.md`, `.yaml`) ubicados en las mismas rutas que tendrían en el repo, más un mensaje corto: qué hicieron, cómo se corre, qué quedó pendiente y qué decisiones tomaron. Lo que sea solo texto puede ir en un Google Doc.
- **Punto de partida:** la última versión de `main` (hacer `git pull` antes de empezar).
- **No tocar:** `notebooks/`, `src/modelo.py`, `src/eda.py`, `src/enriquecimiento.py` ni los archivos que ya existen en `data/processed/`. Las salidas nuevas van a archivos nuevos con `link` como clave.
- **Datos externos:** siempre un snapshot en `data/external/` con la fecha de descarga anotada en la ficha, y el código tiene que correr sin red si el snapshot está.
- **Configuración:** nada de rutas absolutas. Las rutas y parámetros van en `config/config.yaml` (o en `config/costos.yaml` para la Tarea 3), como el resto del pipeline.
- **Estilo:** Python con comillas dobles, nombres en castellano como el resto de `src/`, docstring corta en cada función. Dependencias nuevas en `requirements.txt` con versión fijada.
- **Textos:** castellano rioplatense, oraciones cortas, cifras con fuente. Sin rayas largas en la prosa.
- **Dudas:** si algo de la consigna no cierra con lo que encuentran en los datos, avisen antes de decidir por su cuenta.